



Programación Web 3

UNLaM - Tecnicatura en Desarrollo Web

Trabajo Práctico de Investigación

Título: SignalR



Integrantes	1
Objetivo	2
Situación Actual	2
Desarrollo de la Investigación	3
Introducción a SignalR	3
Arquitectura y funcionamiento	4
Componentes de la arquitectura	4
Funcionamiento	5
Componentes principales	5
Hub	5
HubConnection	6
Métodos del servidor	7
Métodos del cliente	7
Gestión de conexiones	7
Protocolos de transporte	8
Implementación de SignalR	9
Tipos de comunicación	13
Casos de uso	15
Aplicación práctica desarrollada	15
Conclusiones	16
Referencias/Bibliografía	16



Integrantes

Cristian Chazarreta

Elias Tucci

Franco Vargas

Jesica Salva

Martin Boga

Mauricio Galvan

Objetivo

El presente trabajo tiene como objetivo investigar la tecnología SignalR, una biblioteca de ASP.NET Core que permite implementar comunicación en tiempo real entre clientes y servidores. Para ello, se analizarán su arquitectura, el funcionamiento de los Hubs, los protocolos de transporte, la gestión de conexiones y los distintos mecanismos de envío de mensajes que ofrece la tecnología.

Como aplicación práctica, se desarrolló un **marketplace de compra, venta e intercambio de instrumentos musicales, clases de música, shows y equipos de audio**. En este sistema, los usuarios pueden publicar productos o servicios y recibir ofertas de otros usuarios mediante comentarios. Cuando el propietario de una publicación considera que una oferta resulta de su interés, puede iniciar un chat privado con el usuario que la realizó, utilizando SignalR para establecer una comunicación en tiempo real.

A través de esta implementación se busca demostrar la aplicación de SignalR en un caso de uso real, analizando sus ventajas frente al modelo tradicional basado en solicitudes HTTP y su integración con ASP.NET Core y Entity Framework Core para el desarrollo de aplicaciones interactivas.

Situación Actual

SignalR es una biblioteca de ASP.NET Core desarrollada por Microsoft que permite implementar comunicación en tiempo real entre clientes y servidores. Actualmente es una de las tecnologías más utilizadas dentro de la **plataforma .NET** para aplicaciones que requieren la actualización inmediata de la información, sin necesidad de que el cliente realice consultas repetitivas al servidor.

Esta tecnología no es reciente. Su primera versión fue presentada para ASP.NET Framework y, con la evolución de la plataforma hacia ASP.NET Core, fue rediseñada para adaptarse a la nueva arquitectura, mejorando su rendimiento, escalabilidad y compatibilidad con los estándares actuales de comunicación. En la actualidad, SignalR forma parte de ASP.NET Core y continúa siendo una solución ampliamente utilizada para desarrollar aplicaciones en tiempo real.



En las versiones actuales de ASP.NET Core (incluyendo .NET 8, .NET 9 y .NET 10), SignalR ofrece una integración sencilla con aplicaciones web, APIs y servicios backend. La biblioteca utiliza WebSockets como protocolo principal para lograr una comunicación bidireccional de baja latencia y, cuando este protocolo no está disponible, cambia automáticamente a otros mecanismos compatibles, como Server-Sent Events o Long Polling.

En el desarrollo de software moderno, SignalR es ampliamente utilizado para implementar funcionalidades como chats en tiempo real, notificaciones, paneles de monitoreo, aplicaciones colaborativas, juegos multijugador, sistemas de seguimiento de pedidos y plataformas financieras que requieren la actualización inmediata de datos.

Su integración con ASP.NET Core, junto con el soporte para autenticación, autorización, escalabilidad y comunicación entre múltiples clientes, lo convierte en una solución robusta para desarrollar aplicaciones interactivas dentro de la **plataforma .NET**.

Desarrollo de la Investigación

Introducción a SignalR

SignalR es una biblioteca de código abierto para ASP.NET Core, desarrollada por Microsoft, que facilita la comunicación en tiempo real entre el servidor y los clientes. Permite incorporar funcionalidades en tiempo real a las aplicaciones web, haciendo posible que el servidor envíe actualizaciones a los clientes de forma inmediata, sin que estos deban actualizar la página o realizar consultas constantes al servidor.

Gracias a esta tecnología, los desarrolladores pueden crear aplicaciones más dinámicas e interactivas, como chats, sistemas de notificaciones, aplicaciones colaborativas, paneles de monitoreo y juegos en línea.

A diferencia del modelo tradicional basado en solicitudes HTTP, SignalR permite mantener una comunicación continua entre el servidor y los clientes, haciendo posible que la información se actualice en tiempo real.

Modelo Tradicional HTTP	SignalR
El cliente inicia la comunicación enviando una solicitud al servidor.	El cliente establece una conexión y el servidor puede enviar información cuando ocurre un cambio.
Cada actualización requiere una nueva solicitud HTTP.	La información se actualiza automáticamente en tiempo real.
El usuario puede tener que actualizar la página para ver cambios.	No es necesario actualizar la página.
La comunicación es puntual: solicitud y respuesta	La comunicación es continua mientras la conexión permanezca activa.
Es adecuado para aplicaciones donde la información cambia poco.	Es ideal para aplicaciones que requieren información en tiempo real, como chats y notificaciones.



Arquitectura y funcionamiento

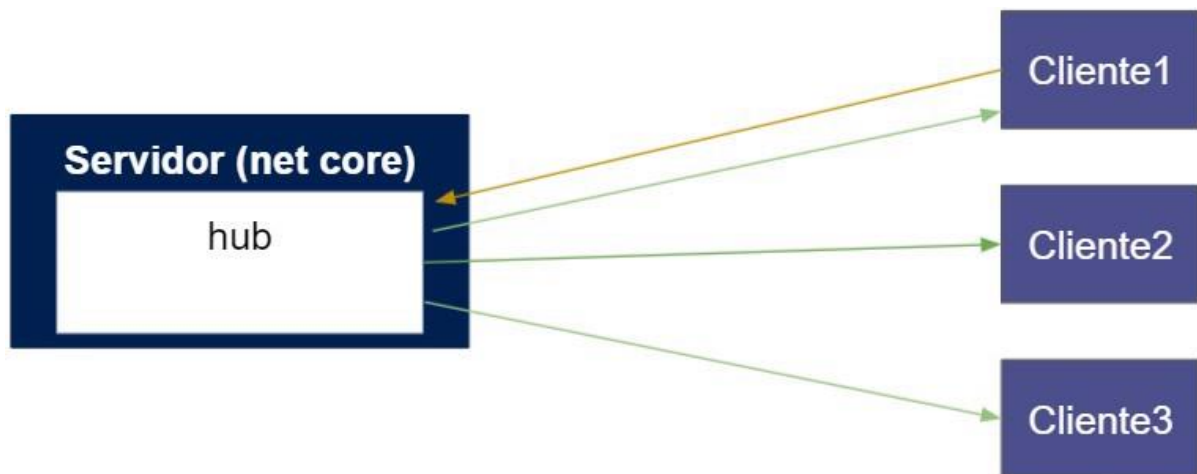
La arquitectura de SignalR está basada en el modelo cliente-servidor y permite establecer una comunicación bidireccional en tiempo real entre ambos. A diferencia del modelo HTTP tradicional, donde el cliente debe enviar una solicitud para obtener información, SignalR mantiene una conexión activa que permite al servidor enviar datos a los clientes cuando ocurre un evento.

Para lograr esta comunicación, SignalR utiliza tres componentes principales: el cliente, el servidor y el Hub. Cada uno cumple una función específica para garantizar el intercambio de mensajes de forma rápida y eficiente.

Componentes de la arquitectura

- **Cliente:** El cliente es la aplicación que se conecta al servidor para enviar y recibir información en tiempo real. Puede tratarse de una aplicación web desarrollada con JavaScript, Angular, React, Blazor o incluso una aplicación móvil o de escritorio compatible con SignalR.
Una vez establecida la conexión con el Hub, el cliente puede invocar métodos del servidor y recibir mensajes enviados por este sin necesidad de realizar nuevas solicitudes HTTP.
- **Servidor:** El servidor es la aplicación ASP.NET Core que aloja SignalR y administra las conexiones de los clientes. Su función es recibir las solicitudes, procesarlas y enviar la información al cliente, a un usuario específico o a un grupo de usuarios. Además, el servidor gestiona las conexiones activas y coordina el intercambio de mensajes entre los distintos clientes conectados.
- **El Hub** es el componente principal de SignalR y actúa como intermediario entre el servidor y los clientes. A través del Hub, los clientes pueden invocar métodos del servidor y el servidor puede enviar mensajes a uno o varios clientes.
El Hub simplifica la comunicación en tiempo real, ya que proporciona una interfaz para gestionar conexiones y distribuir mensajes sin que el desarrollador deba implementar manualmente los mecanismos de comunicación.

En esta arquitectura, todos los clientes establecen una conexión con el **Hub** alojado en el servidor. Cuando un cliente envía un mensaje, este llega al Hub, que se encarga de procesarlo y distribuirlo al destinatario correspondiente, ya sea un cliente específico, un usuario, un grupo o todos los clientes conectados.



Funcionamiento

El funcionamiento de SignalR se basa en una comunicación continua entre el cliente y el servidor. Una vez que el cliente establece una conexión con el Hub, ambos pueden intercambiar información en tiempo real sin necesidad de realizar nuevas solicitudes HTTP para cada mensaje.

El proceso de comunicación ocurre de la siguiente manera:

1. **El cliente inicia la conexión.** Al acceder a la aplicación, el cliente establece una conexión con el Hub de SignalR alojado en el servidor.
2. **El servidor acepta la conexión.** Una vez establecida, el servidor registra al cliente y queda preparado para enviar y recibir mensajes.
3. **El cliente envía una solicitud o mensaje.** Cuando el usuario realiza una acción, como enviar un mensaje en un chat, el cliente invoca un método del Hub.
4. **El Hub procesa la solicitud.** El Hub recibe la información y determina a quién debe enviarla. Puede reenviarla a todos los clientes conectados, a un usuario específico o a un grupo de usuarios.
5. **El servidor distribuye el mensaje.** SignalR envía la información a los destinatarios correspondientes utilizando la conexión que permanece activa.
6. **Los clientes reciben la actualización en tiempo real.** Los usuarios visualizan el nuevo mensaje o la actualización inmediatamente, sin necesidad de actualizar la página ni realizar una nueva solicitud al servidor.

Componentes principales

Hub

El **Hub** es el componente central de SignalR y el encargado de administrar la comunicación entre el servidor y los clientes conectados. A través del Hub, los clientes pueden invocar métodos del servidor y el servidor puede enviar mensajes a uno o varios clientes.

`using Microsoft.AspNetCore.SignalR;`



```
public class ChatHub : Hub
{
    //Codigo
}
```

En este ejemplo se crea un Hub llamado ChatHub, que será el punto de comunicación entre el servidor y los clientes.

HubConnection

HubConnection representa la conexión que establece el cliente con el Hub del servidor. Gracias a esta conexión, el cliente puede enviar mensajes al servidor y recibir actualizaciones en tiempo real.

```
const connection = new signalR.HubConnectionBuilder()
    .withUrl("https://localhost:5001/chatHub")
    .build();
```

```
await connection.start();
```

En este caso, el cliente crea una conexión con el Hub ubicado en /chatHub. Una vez iniciada, podrá intercambiar mensajes con el servidor.

HubContext

IHubContext permite enviar mensajes a los clientes desde cualquier parte de la aplicación, sin necesidad de estar dentro del Hub.

```
public class ChatController : Controller
{
    private readonly IHubContext<ChatHub> _hubContext;

    public ChatController(IHubContext<ChatHub> hubContext)
    {
        _hubContext = hubContext;
    }

    public async Task<IActionResult> Notificar()
    {
        await _hubContext.Clients.All.SendAsync(
            "RecibirMensaje",
            "Sistema",
            "Nuevo mensaje");

        return Ok();
    }
}
```



```
}  
}
```

Métodos del servidor

Son los métodos definidos dentro del Hub que pueden ser invocados por los clientes. Generalmente contienen la lógica necesaria para procesar una solicitud o distribuir un mensaje.

```
public class ChatHub : Hub  
{  
    public async Task EnviarMensaje(string usuario, string mensaje)  
    {  
        await Clients.All.SendAsync(  
            "RecibirMensaje",  
            usuario,  
            mensaje);  
    }  
}
```

Cuando un cliente invoca el método `EnviarMensaje`, el Hub recibe los datos y los envía a todos los clientes conectados.

Métodos del cliente

Los métodos del cliente son aquellos que el servidor invoca para enviar información a las aplicaciones conectadas. En el cliente, estos métodos se registran mediante `connection.on()`, indicando el nombre del método que será ejecutado cuando el servidor envíe un mensaje.

Ejemplo de js :

```
const conexion = new signalR.HubConnectionBuilder()  
    .withUrl("/hubs/chat")  
    .build();  
  
conexion.on("NuevaNotificacion", function (data) {  
    agregarNotificacion(data);  
});
```

Gestión de conexiones

ConnectionId



Cada cliente que se conecta a SignalR recibe un `ConnectionId`, que es un identificador único para esa conexión. El servidor puede utilizarlo para identificar al cliente durante la comunicación.

```
public override async Task OnConnectedAsync()
{
    var id = Context.ConnectionId;

    await base.OnConnectedAsync();
}
```

OnConnectedAsync()

Es un método que se ejecuta cuando un cliente establece una conexión con el Hub. Se utiliza para:

- Registrar usuarios.
- Agregarlos a grupos.
- Mostrar que un usuario se conectó.

```
public override async Task OnConnectedAsync()
{
    Console.WriteLine( $"Cliente conectado: {Context.ConnectionId}");

    await base.OnConnectedAsync();
}
```

OnDisconnectedAsync()

Se ejecuta cuando un cliente pierde la conexión o cierra la aplicación.

Se utiliza para:

- Eliminar usuarios conectados.
- Sacarlos de grupos.
- Liberar recursos.

```
public override async Task OnDisconnectedAsync(Exception? exception)
{
    Console.WriteLine($"Cliente desconectado: {Context.ConnectionId}");
    await base.OnDisconnectedAsync(exception);
}
```

Protocolos de transporte

Los protocolos de transporte son los mecanismos que utiliza SignalR para intercambiar información entre el servidor y los clientes. Su función es garantizar que los mensajes puedan enviarse y recibirse de forma eficiente y en tiempo real.

SignalR selecciona automáticamente el protocolo más adecuado según las capacidades del cliente, del servidor y del navegador. El orden de preferencia es: **WebSockets**, **Server-Sent Events (SSE)** y, como última alternativa, **Long Polling**.



WebSockets

WebSockets es el protocolo de transporte preferido por SignalR, ya que permite establecer una conexión bidireccional y permanente entre el cliente y el servidor. Una vez creada la conexión, ambas partes pueden enviar y recibir mensajes en cualquier momento sin necesidad de abrir nuevas conexiones.

Este protocolo ofrece la menor latencia y el mejor rendimiento, por lo que es la opción recomendada para aplicaciones que requieren comunicación en tiempo real, como chats, videojuegos en línea y sistemas de notificaciones

Server-Sent Events o Eventos enviados por el servidor (SSE)

Server-Sent Events es un mecanismo que permite al servidor enviar información al cliente mediante una conexión HTTP que permanece abierta.

A diferencia de WebSockets, la comunicación es unidireccional: el servidor puede enviar datos al cliente, pero el cliente debe seguir utilizando solicitudes HTTP tradicionales para enviar información al servidor.

SignalR utiliza este protocolo únicamente cuando WebSockets no está disponible.

Long Polling o Sondeo Largo

Long Polling es el mecanismo de respaldo utilizado por SignalR cuando no es posible emplear WebSockets ni Server-Sent Events.

En este modelo, el cliente envía una solicitud HTTP al servidor y este mantiene la conexión abierta hasta que haya información disponible o se alcance un tiempo de espera. Una vez enviada la respuesta, el cliente realiza inmediatamente una nueva solicitud para continuar recibiendo actualizaciones.

Aunque permite simular una comunicación en tiempo real, genera un mayor número de solicitudes HTTP y ofrece un rendimiento inferior al de los otros protocolos.

La técnica de transporte o protocolo de transporte no se requiere elegir manualmente. SignalR detecta automáticamente cuál puede utilizar según las capacidades del cliente y del servidor. Si WebSockets está disponible, será el protocolo seleccionado; de lo contrario, SignalR utilizará Server-Sent Events y, como última alternativa, Long Polling.

Implementación de SignalR

En la aplicación desarrollada, SignalR se implementó para permitir la comunicación en tiempo real entre los usuarios del marketplace. Su uso hizo posible actualizar las ofertas, enviar notificaciones y establecer conversaciones privadas sin necesidad de recargar la página.

La implementación se realizó siguiendo los siguientes pasos.

1. Agregar la dependencia de SignalR:

En un proyecto ASP.NET Core, SignalR se incorpora mediante el paquete oficial de Microsoft:



```
dotnet add package Microsoft.AspNetCore.SignalR
```

Este paquete proporciona las clases y componentes necesarios para crear Hubs y administrar la comunicación en tiempo real.

2. Registrar el servicio SignalR:

Una vez instalado el paquete, se debe registrar el servicio de SignalR en el archivo **Program.cs**.

```
builder.Services.AddSignalR();
```

Con esta configuración, la aplicación habilita los servicios necesarios para utilizar SignalR.

3. Crear un Hub:

El siguiente paso consiste en crear una clase que herede de Hub. Esta clase será el punto de comunicación entre el servidor y los clientes.

Dentro del Hub se implementan los métodos que podrán ser invocados por los clientes y se administra el envío de mensajes en tiempo real.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;
using Microsoft.EntityFrameworkCore;

namespace WebApplication1.Hubs;

[Authorize]
public class OfertaHub : Hub
{
    private readonly MusicTradeDbContext _db;

    public OfertaHub(MusicTradeDbContext db)
    {
        _db = db;
    }
}
```

4. Configurar la ruta del hub:

Luego se debe indicar a ASP.NET Core la dirección mediante la cual los clientes podrán conectarse al Hub en el Program.cs.

```
app.MapHub<OfertaHub>("/hubs/oferta");
app.MapHub<ChatHub>("/hubs/chat");
```



Estas rutas permiten que los clientes conozcan la dirección a la que deben conectarse para comunicarse con cada Hub.

5. Crear la conexión con el hub

Una vez configurado el Hub en el servidor, el cliente debe crear una conexión para poder intercambiar información en tiempo real. Para ello, SignalR proporciona la clase *HubConnectionBuilder*, que permite configurar cómo se establecerá la comunicación con el servidor.

En la aplicación desarrollada, la conexión se crea de la siguiente manera:

```
var conexion = new signalR.HubConnectionBuilder()
    .withUrl("/hubs/oferta")
    .withAutomaticReconnect()
    .build();
```

6. Iniciar la conexión

Una vez creada la conexión, el cliente debe iniciarla para comenzar a comunicarse con el Hub.

El método `start()` establece la conexión entre el cliente y el servidor. Cuando la conexión se realiza correctamente, el cliente invoca el método `JoinGroup()` del Hub para unirse al grupo correspondiente a la publicación que está visualizando. A partir de ese momento, podrá recibir en tiempo real las nuevas ofertas y actualizaciones enviadas por el servidor.

```
conexion.start().then(function () {
    return conexion.invoke("JoinGroup", publicacionId);
}).catch(function (err) {
    console.error("Error ofertas SignalR:", err.toString());
});
```

7. Implementar los métodos del Hub.

En este paso se implementan los métodos del servidor que gestionarán la comunicación entre los clientes.

En este proyecto, el método `EnviarOferta` recibe la oferta enviada por un usuario, la almacena mediante Entity Framework Core y posteriormente la distribuye a los clientes correspondientes.



```
public class OfertaHub : Hub
{
    public async Task EnviarOferta(int publicacionId, string mensaje)
    {
        Estado = EstadoOferta.Pendiente;

        _db.Ofertas.Add(oferta);
        await _db.SaveChangesAsync();

        await Clients.Group($"publicacion-{publicacionId}").SendAsync("RecibirOferta", new
        {
            oferta.Id,
            oferta.UsuarioId,
            UsuarioNombre = $"{usuario.Nombre} {usuario.Apellido}",
            oferta.Mensaje,
            oferta.FechaCreacion,
            oferta.PublicacionId
        });
    }
}
```

8. Implementación de los métodos del cliente

Los métodos del cliente son registrados mediante `connection.on()`. Estos métodos se ejecutan automáticamente cuando el servidor envía un mensaje utilizando `SendAsync()`. En la aplicación desarrollada se utilizan para mostrar nuevas ofertas y notificaciones sin necesidad de actualizar la página.

```
conexion.on("RecibirOferta", function (oferta) {
    agregarOferta(oferta);
});
```

9. Enviar mensaje

En la aplicación desarrollada, el envío de una oferta sigue un flujo compuesto por tres etapas: el cliente invoca un método del servidor, el servidor procesa la información y, finalmente, envía el resultado a los clientes correspondientes.

9.1. El cliente invoca al metodo del servidor

Cuando el usuario escribe una oferta y presiona el botón **Enviar**, el cliente invoca el método `EnviarOferta()` definido en el Hub.

9.2. El servidor procesa la información y envia el mensaje

Una vez recibida la solicitud, el método `EnviarOferta()` del Hub hace lo que tiene que hacer y luego la distribuye a todos los clientes que pertenecen al grupo de la publicación

9.3. El cliente recibe el mensaje

Los clientes registran previamente el método que será ejecutado cuando el servidor envíe una nueva oferta.

10. Gestion de grupos



Como vimos previamente, SignalR administra cada conexión mediante un identificador único denominado **ConnectionId**. Además, permite organizar las conexiones en **grupos**, facilitando el envío de mensajes únicamente a los usuarios que corresponden.

En la aplicación desarrollada se utilizaron grupos para organizar las conversaciones, las publicaciones y las notificaciones personales.

```
public async Task JoinGroup(int publicacionId)
{
    await Groups.AddToGroupAsync(Context.ConnectionId, $"publicacion-{publicacionId}");
}
```

11. Gestion de conexiones

Cuando un usuario establece una conexión con el Hub, SignalR ejecuta automáticamente el método `OnConnectedAsync()`.

```
public override async Task OnConnectedAsync()
{
    if (Context.User?.Identity?.IsAuthenticated == true)
    {
        var usuarioId = int.Parse(Context.User.FindFirstValue(ClaimTypes.NameIdentifier!));
        await Groups.AddToGroupAsync(Context.ConnectionId, $"user-{usuarioId}");
    }
    await base.OnConnectedAsync();
}
```

Nuestra aplicación también implementa la **reconexión automática** utilizando `withAutomaticReconnect()`. Cuando la conexión se restablece, el cliente vuelve a incorporarse al grupo correspondiente.

Al producirse una reconexión, SignalR genera un nuevo `ConnectionId`. Como la nueva conexión ya no pertenece a los grupos anteriores, el cliente vuelve a ejecutar `JoinGroup()` para reincorporarse al grupo de la publicación y continuar recibiendo las actualizaciones en tiempo real.

Tipos de comunicación

Una de las principales características de SignalR es la posibilidad de enviar mensajes a distintos destinatarios según las necesidades de la aplicación. El servidor puede enviar información a todos los clientes conectados, a una conexión específica, a un usuario determinado, a un grupo de usuarios o incluso excluir determinadas conexiones.

Esta flexibilidad permite desarrollar aplicaciones como chats privados, chats grupales, sistemas de notificaciones y aplicaciones colaborativas.



- **Comunicación con todos (Clients.All):** Envía un mensaje a **todos los clientes conectados**. Se lo usa en casos como chat grupal, notificaciones generales, avisos del sistema.

Ejemplo:

```
await Clients.All.SendAsync(  
    "RecibirMensaje",  
    usuario,  
    mensaje);
```

- **Comunicación con un cliente (Clients.Client):** Envía un mensaje únicamente a una conexión específica, identificada mediante un ConnectionId. Este tipo de comunicación se lo usa en casos para confirmar una operación, mostrar un mensaje solamente al cliente que realizó una acción.

Ejemplo

```
await Clients.Client(connectionId)  
    .SendAsync(  
        "RecibirMensaje",  
        mensaje);
```

- **Comunicación con un usuario (Clients.User):** Envía un mensaje a un usuario autenticado utilizando su identificador de usuario. Si el usuario tiene varias conexiones activas, todas recibirán el mensaje. Esto significa que SignalR enviará el mensaje a **todas las conexiones asociadas** a ese usuario. Se lo usa en casos como chat privado, notificaciones personales, o mensajes directos

Ejemplo

```
await Clients.User(userId)  
    .SendAsync(  
        "RecibirMensaje",  
        mensaje);
```

- **Comunicación con un grupo (Clients.Group):** Permite enviar mensajes a un conjunto de usuarios agrupados bajo un mismo nombre. Se lo usa para chats grupales, equipos de trabajo, salas de juego o canales de discusión.

Ejemplo

```
await Clients.Group("Músicos")  
    .SendAsync(  
        "RecibirMensaje",  
        mensaje);
```

En este ejemplo sería que solo los clientes que pertenecen al grupo "Músicos" recibirán el mensaje.

- **Comunicación excluyendo clientes (Clients.AllExcept):** Envía un mensaje a todos los clientes conectados, excepto a uno o varios indicados. Se lo suele usar



para actualizar a todos menos al usuario que realizó la acción o para evitar enviar mensajes duplicados.

En este ejemplo todos los clientes recibirán el mensaje excepto la conexión indicada:

```
await Clients.AllExcept(connectionId)
    .SendAsync(
        "RecibirMensaje",
        mensaje);
```

Casos de uso

SignalR se utiliza en aplicaciones que requieren comunicación en tiempo real y actualización inmediata de la información.

Algunos casos de uso frecuentes son:

- **Chats en tiempo real:** permiten la comunicación instantánea entre usuarios, tanto en chats privados como grupales.
- **Sistemas de notificaciones:** envían alertas o avisos sin necesidad de actualizar la página.
- **Aplicaciones colaborativas:** varios usuarios pueden trabajar simultáneamente sobre la misma información y ver los cambios en tiempo real.
- **Paneles de monitoreo (dashboards):** muestran métricas, estados o eventos actualizados constantemente.
- **Videojuegos multijugador:** permiten sincronizar acciones y eventos entre los jugadores conectados.
- **Seguimiento de pedidos o servicios:** informan cambios de estado de manera inmediata.
- **Marketplaces y plataformas de intercambio:** facilitan la comunicación directa entre compradores y vendedores mediante chats en tiempo real.

Además, SignalR puede integrarse con servicios de Inteligencia Artificial para desarrollar asistentes virtuales o chatbots que respondan en tiempo real a los usuarios.

Aplicación práctica desarrollada

Como aplicación práctica de la investigación se desarrolló un **marketplace de compra, venta e intercambio**, implementado con **ASP.NET Core .NET 10**, **Entity Framework Core** y **SignalR**.

El sistema permite que los usuarios registrados publiquen distintos tipos de anuncios, como instrumentos musicales, clases, shows y equipos de audio. Cada publicación puede recibir ofertas o propuestas realizadas por otros usuarios mediante comentarios.



Cuando el propietario de una publicación considera que una oferta resulta de su interés, puede iniciar un **chat privado** con el usuario que la realizó. Esta comunicación se lleva a cabo mediante SignalR, lo que permite que ambos intercambien mensajes en tiempo real sin necesidad de actualizar la página.

Si ambas partes llegan a un acuerdo, el propietario puede cerrar la publicación. A partir de ese momento, la publicación deja de recibir nuevas ofertas, indicando que la negociación ha finalizado.

Para implementar estas funcionalidades se utilizaron distintos grupos de SignalR:

- **publicacion-{id}**: permite actualizar en tiempo real las ofertas realizadas sobre una publicación para todos los usuarios que la están visualizando.
- **conversacion-{id}**: representa cada chat privado entre el propietario de la publicación y el usuario con quien decide negociar.
- **user-{id}**: se utiliza para enviar notificaciones personales a los usuarios conectados.

Esta implementación permitió integrar la persistencia de datos mediante Entity Framework Core con la comunicación en tiempo real proporcionada por SignalR, ofreciendo una experiencia de usuario más dinámica e interactiva.

Conclusiones

La investigación y el desarrollo de la aplicación permitieron comprender el funcionamiento de SignalR y su importancia para implementar comunicación en tiempo real en aplicaciones desarrolladas con ASP.NET Core.

Durante el proyecto se comprobó que SignalR facilita el intercambio de información entre el servidor y los clientes, ofreciendo una solución eficiente para funcionalidades como chats, notificaciones y actualizaciones instantáneas, sin necesidad de que los usuarios recarguen la página.

En la implementación práctica se analizaron las distintas formas de envío de mensajes que ofrece SignalR. Se optó por utilizar **grupos (Clients.Group)** para administrar las conversaciones privadas, las notificaciones y las actualizaciones de las publicaciones. Esta decisión permitió organizar la comunicación de forma más flexible y escalable, ya que cada conversación y cada publicación se representan mediante un grupo independiente.

Si bien SignalR también ofrece mecanismos como **Clients.Client** y **Clients.User**, el uso de grupos resultó más adecuado para los requerimientos del proyecto. A diferencia de **Clients.Client**, que envía mensajes a una única conexión, los grupos permiten que un mismo usuario participe desde varios dispositivos o pestañas sin perder mensajes. Además, este enfoque facilita la incorporación de nuevas funcionalidades, como la posibilidad de ampliar una conversación privada a un chat grupal sin modificar la arquitectura de la aplicación.

En conclusión, SignalR demostró ser una herramienta robusta, flexible y de fácil integración con ASP.NET Core y Entity Framework Core. Su utilización permitió desarrollar un marketplace con comunicación en tiempo real, mejorando la interacción entre los usuarios y ofreciendo una base sólida para futuras ampliaciones del sistema.



Referencias/Bibliografía

Microsoft. (s. f.). *ASP.NET Core SignalR*

<https://learn.microsoft.com/es-es/aspnet/core/signalr/introduction?view=aspnetcore-10.0>

Microsoft. (s. f.). *ASP.NET Core SignalR Hubs*.

<https://learn.microsoft.com/es-es/aspnet/core/signalr/hubs?view=aspnetcore-10.0>